UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

FACULTAD DE INGENIERÍA ELÉCTRICA, ELECTRÓNICA, INFORMÁTICA Y MECÁNICA

ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS



INFORME DE GUÍA DE LABORATORIO N.º 03

PRÁCTICA: MEDICIÓN DE DISTANCIA CON ESP32 Y SENSOR HC-SR04

IMPLEMENTACIÓN, PROGRAMACIÓN, CALIBRACIÓN Y PRUEBA EXPERIMENTAL

Asignatura: ROBÓTICA

Estudiante: Noe Franklin Choquenaira Quispe

Código: 133962

Semestre académico: 2026-II

Docente: Jose Mauro Pillco Quispe

Cusco – Perú

2026

1. Introducción

El presente informe documenta el proceso real de construcción y prueba de un sistema de medición de distancia basado en un ESP32 y un sensor ultrasónico HC-SR04. El trabajo se desarrolló de manera progresiva: primero se realizó el montaje físico sobre el protoboard, luego se establecieron las conexiones de alimentación y señales, se programó el ESP32 desde Arduino IDE y finalmente se efectuaron pruebas de medición y calibración.

El objetivo del documento es registrar cómo se construyó el prototipo, qué conexiones se utilizaron, qué código permitió obtener las mediciones y cómo se verificó el resultado mediante un objeto y una regla. Las fotografías corresponden al proceso y a las pruebas realizadas durante la implementación.

2. Objetivos

2.1. Objetivo general

Implementar y comprobar un sistema de medición de distancia con ESP32 y HC-SR04, mostrando el resultado en el Monitor Serial del Arduino IDE.

2.2. Objetivos específicos

- Montar el ESP32 y el HC-SR04 sobre un protoboard.
- Identificar y conectar correctamente VCC, GND, TRIG y ECHO.
- Programar la generación del pulso ultrasónico y la lectura del tiempo de retorno.
- Convertir el tiempo de vuelo del ultrasonido a distancia en centímetros.
- Establecer un rango de detección práctico y una condición para objetos demasiado cercanos o lecturas inválidas.
- Verificar la medición utilizando una regla como referencia física.

3. Materiales utilizados

Cuadro 1: Elementos empleados en el montaje.

Elemento	**Uso**
ESP32 Dev Module	Unidad de control y procesamiento.
Sensor HC-SR04	Emisión y recepción de ultrasonido para medir distancia.
Protoboard	Montaje y distribución de conexiones.
Cables Dupont	Interconexión entre ESP32, protoboard y sensor.
Resistencias del montaje	Adaptación de nivel de la señal ECHO para la entrada del ESP32.
Cable USB	Alimentación y programación del ESP32.
Regla	Referencia física para comprobar la distancia medida.

4. Identificación de pines y conexiones

El HC-SR04 posee cuatro conexiones principales: VCC, TRIG, ECHO y GND. En el montaje realizado se utilizaron los GPIO 5 y 18 del ESP32 para las señales de disparo y recepción, respectivamente.

Cuadro 2: Correspondencia de pines utilizada en el programa.

HC-SR04	**ESP32**	**Función**
VCC	5V / VIN	Alimentación del sensor
GND	GND	Referencia común
TRIG	GPIO 5	Pulso de disparo
ECHO	GPIO 18	Retorno del pulso

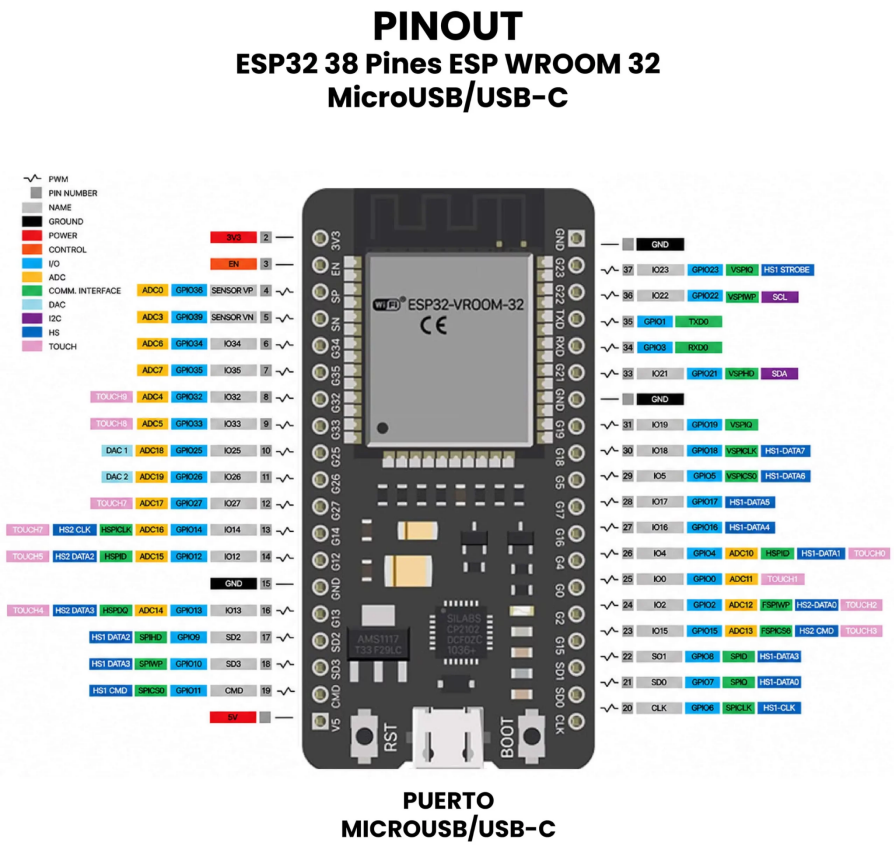


Figura 1: Diagrama de pines del ESP32 utilizado como referencia durante el montaje.

Nota de seguridad eléctrica. La señal ECHO del HC-SR04 puede alcanzar el nivel de alimentación del sensor. Como el ESP32 trabaja con lógica de 3.3 V, el montaje debe adaptar el nivel de ECHO antes de llevarlo al GPIO 18. En las fotografías del prototipo se observa el uso de resistencias para esta adaptación.

5. Proceso de construcción

5.1. Paso 1. Preparación del protoboard y ESP32

Se colocó el ESP32 sobre el protoboard, dejando accesibles sus filas de pines. Se identificaron las líneas de alimentación y se preparó el espacio para ubicar el sensor ultrasónico.

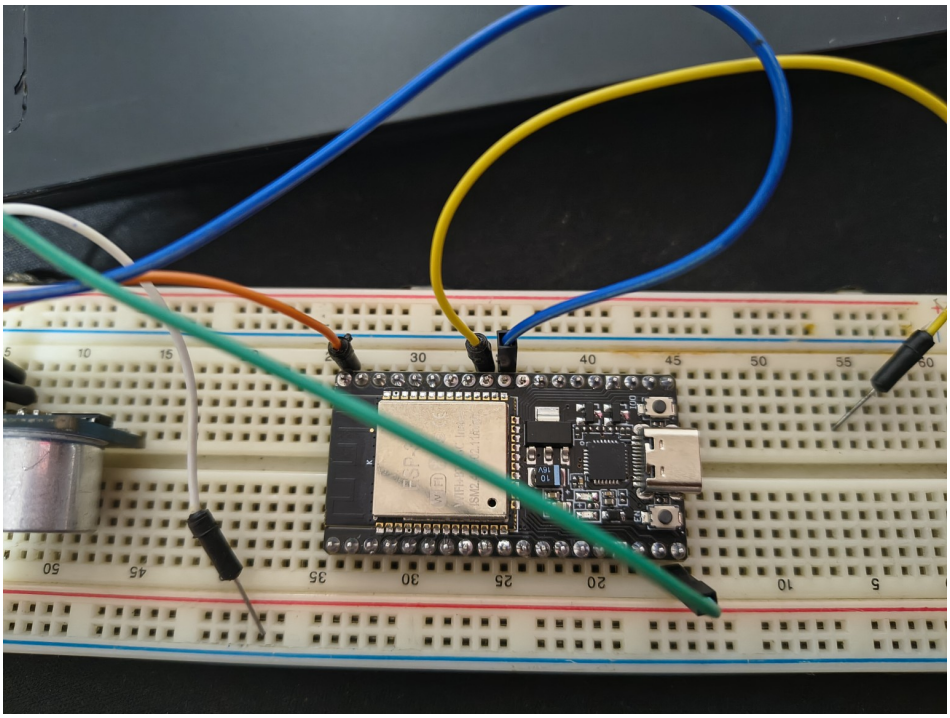


Figura 2: Montaje del ESP32 sobre el protoboard y preparación de las conexiones.

5.2. Paso 2. Colocación del sensor HC-SR04

El HC-SR04 se instaló en el extremo del protoboard para mantener sus dos transductores orientados hacia el objeto que se desea medir. Se identificaron sus cuatro pines: VCC, TRIG, ECHO y GND.

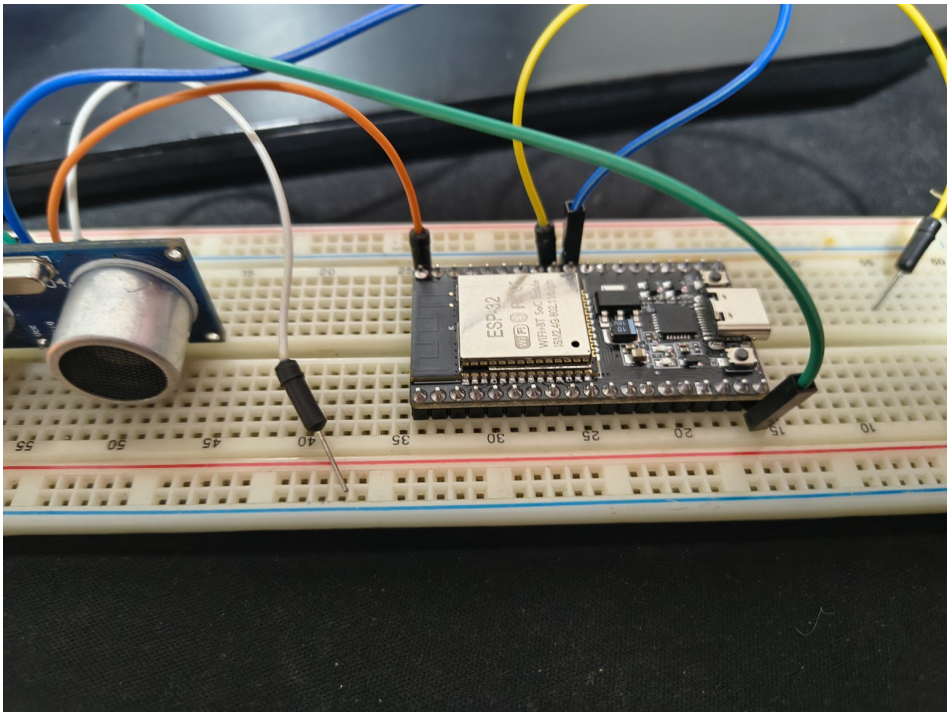


Figura 3: Colocación del sensor HC-SR04 en el protoboard.

5.3. Paso 3. Conexión de alimentación y señales

Se conectó la alimentación, la referencia común y las dos señales de control. TRIG se llevó al GPIO 5 y ECHO al GPIO 18. En la ruta de ECHO se mantuvo la adaptación de nivel mediante las resistencias del montaje.

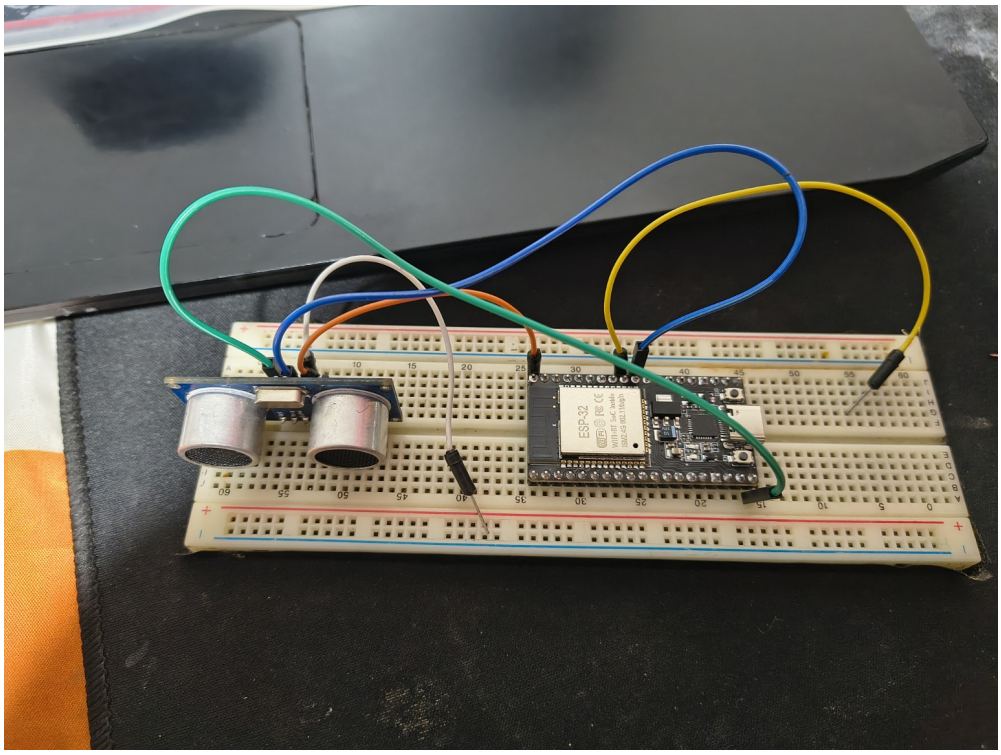


Figura 4: Vista del cableado del ESP32, sensor y protoboard durante el proceso.

5.4. Paso 4. Verificación visual del montaje

Antes de programar, se revisó que las conexiones permanecieran firmes y que el sensor estuviera orientado hacia la zona de medición. Esta comprobación permitió reducir errores producidos por cables sueltos o conexiones cruzadas.

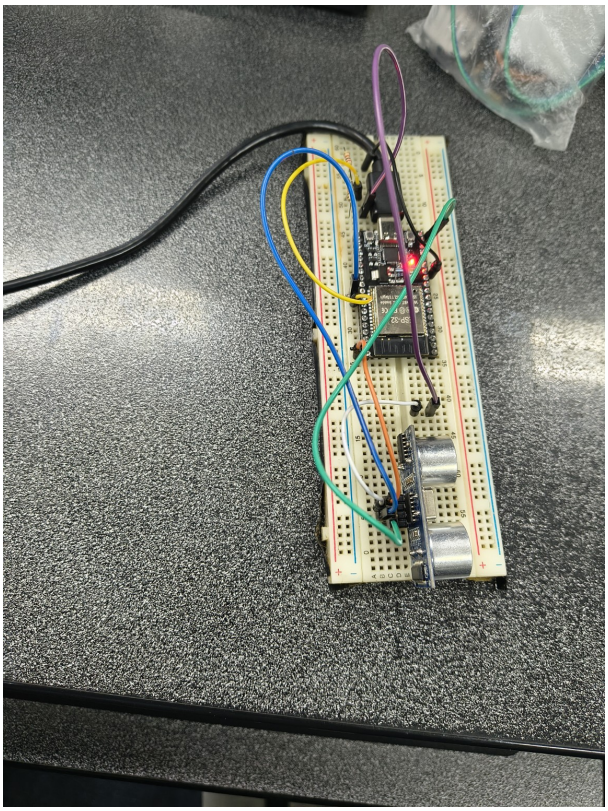


Figura 5: Vista general del montaje físico utilizado para las pruebas.

5.5. Paso 5. Programación en Arduino IDE

Se utilizó Arduino IDE para compilar y cargar el programa en el ESP32. El código genera un pulso de 10 microsegundos en TRIG, mide el tiempo durante el cual ECHO permanece en nivel alto mediante `pulseIn()` y calcula la distancia con la velocidad aproximada del sonido en aire.

La relación empleada es:

$$d = \frac{t\,v}{2}$$

donde t es el tiempo de ida y vuelta, $v \approx 0.0343\ cm/\text{ţ}s$ y el factor 2 representa el recorrido de ida y regreso.

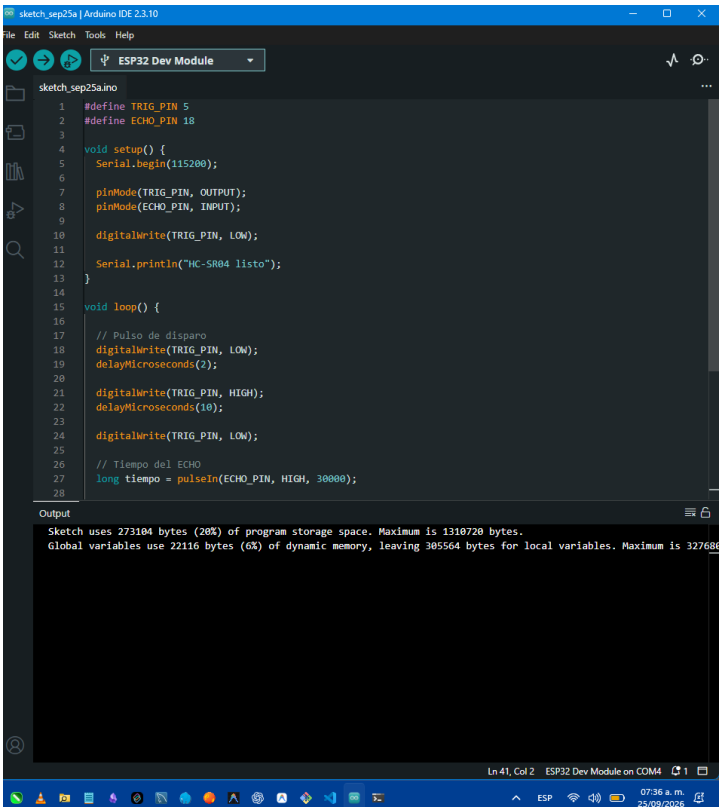


Figura 6: Programación del ESP32 en Arduino IDE durante el desarrollo.

5.6. Paso 6. Carga y ejecución del programa

Una vez compilado el sketch, se cargó en el ESP32 y se abrió el Monitor Serial a 115200 baudios. La salida permite observar la distancia en centímetros y comprobar que el sensor responde de manera repetitiva.

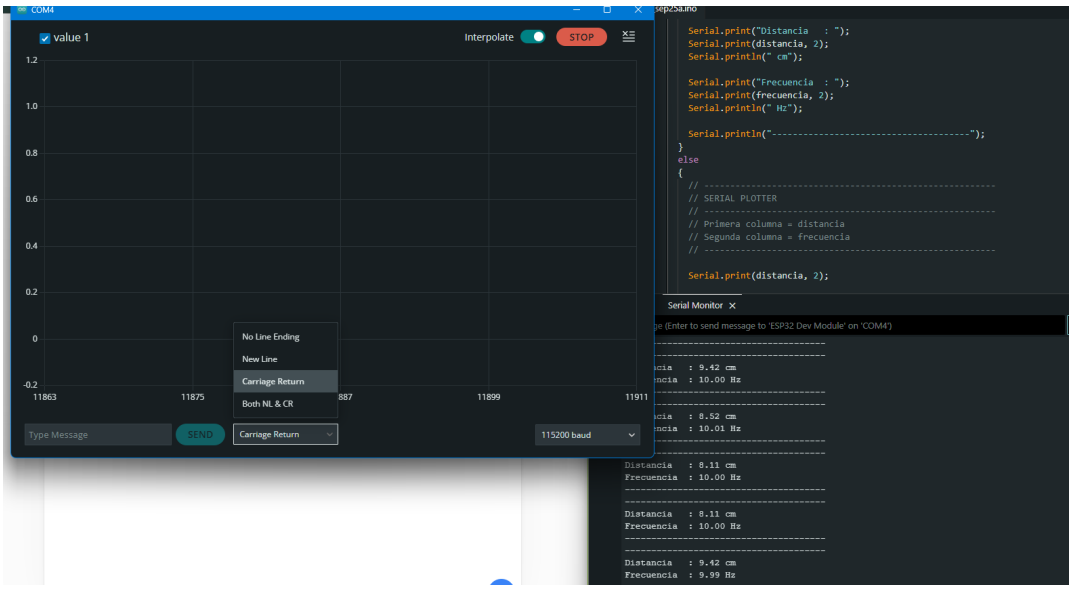


Figura 7: Monitor Serial y código durante la comprobación de las mediciones.

6. Código utilizado

El siguiente programa corresponde a la versión simplificada utilizada para la medición. Se mantiene un intervalo de 100 ms, un tiempo máximo de espera de 30 ms y un rango práctico de detección de hasta 100 cm. Las lecturas demasiado cercanas se tratan como cero para representar la condición de objeto pegado al sensor, mientras que una ausencia de eco se reporta como "SIN OBJETO".

Listing 1: Programa completo para ESP32 + HC-SR04.

```
// ============================================================
// ESP32 + HC-SR04
// Medicion de distancia
// ============================================================

#define TRIG_PIN 5
#define ECHO_PIN 18

const unsigned long INTERVALO = 100; // ms
const unsigned long TIMEOUT = 30000; // us
const float DISTANCIA_CERO = 3.0; // cm
const float DISTANCIA_MAX = 100.0; // cm

float medirDistancia()
{
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);

  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long tiempo = pulseIn(ECHO_PIN, HIGH, TIMEOUT);

  if (tiempo == 0)
    return -1.0;

  float d = (tiempo * 0.0343) / 2.0;

  return d;
}

void setup()
{
  Serial.begin(115200);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  digitalWrite(TRIG_PIN, LOW);

  delay(500);

  Serial.println("HC-SR04 listo");
```

```
}

void loop()
{
  float distancia = medirDistancia();

  if (distancia < 0)
  {
    Serial.println("SIN OBJETO");
  }
  else if (distancia <= DISTANCIA_CERO)
  {
    Serial.println("Distancia: 0.00 cm");
  }
  else if (distancia <= DISTANCIA_MAX)
  {
    Serial.print("Distancia: ");
    Serial.print(distancia, 2);
    Serial.println(" cm");
  }
  else
  {
    Serial.println("SIN OBJETO");
  }

  delay(INTERVALO);
}
```

7. Pruebas y calibración

7.1. Prueba de distancia con objeto

Para verificar el comportamiento del sistema se colocó un objeto delante del HC-SR04 y se utilizó una regla como referencia. La finalidad fue comparar la distancia física con el valor mostrado por el Monitor Serial.

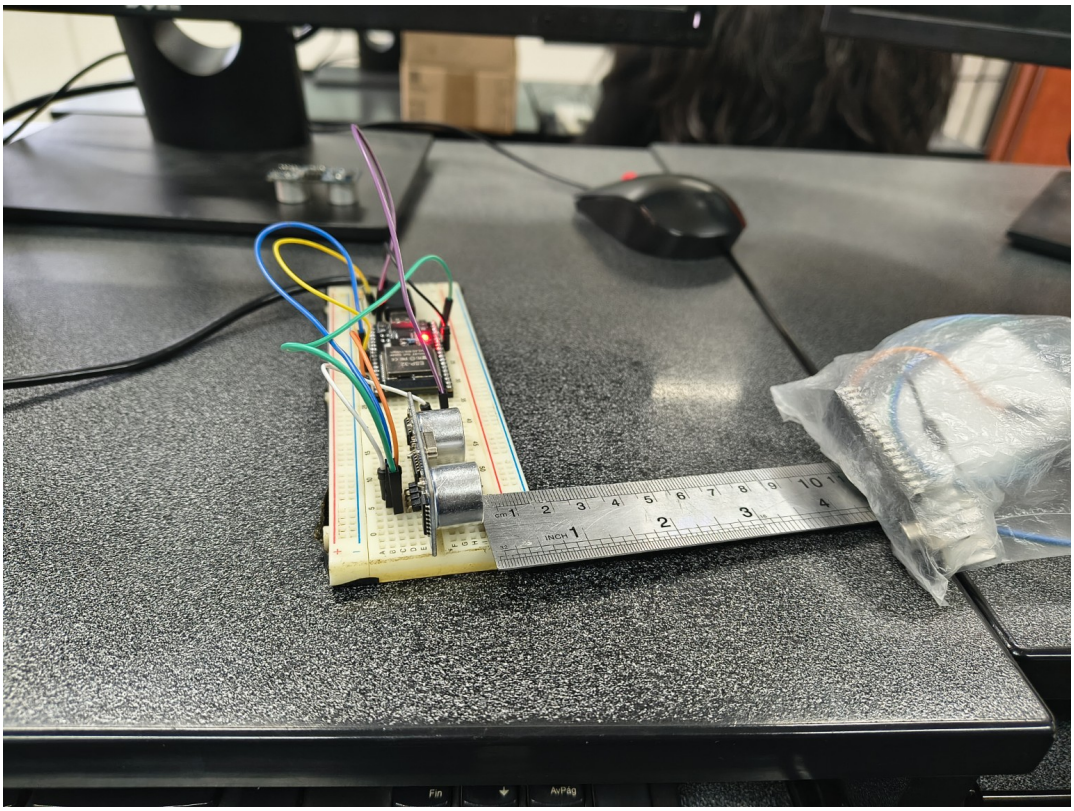


Figura 8: Prueba física: sensor orientado hacia el objeto y regla utilizada como referencia.

7.2. Interpretación de la medición

En la prueba final registrada se observó una lectura cercana a 11 cm, coherente con la posición del objeto respecto al frente del sensor. El valor mostrado por el programa corresponde a la distancia calculada a partir del tiempo de vuelo del ultrasonido.

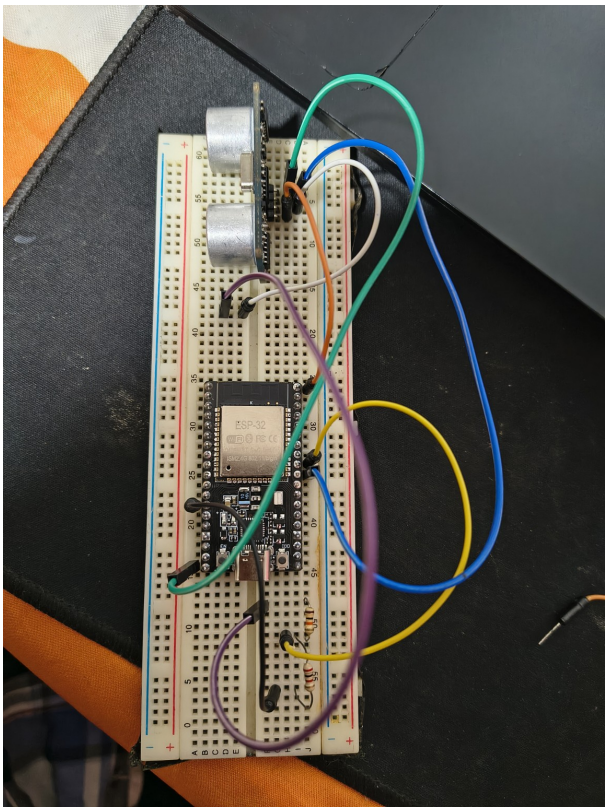


Figura 9: Montaje durante una de las pruebas de medición.

7.3. Comportamiento cuando el objeto está muy cerca

El HC-SR04 tiene una zona mínima de operación. Cuando el objeto se coloca prácticamente pegado a los transductores, la lectura puede dejar de representar correctamente una distancia física de cero. Por ello, el programa establece una zona de tolerancia de 3 cm y presenta 0.00 cm cuando la medición cae dentro de ese umbral.

Esta corrección es una decisión de procesamiento y no significa que el sensor mida físicamente cero centímetros; significa que el sistema interpreta una distancia muy pequeña como condición de objeto pegado al sensor.

7.4. Comportamiento sin objeto

Cuando no existe un eco válido dentro del tiempo de espera, la función `pulseIn()` devuelve cero. El programa identifica esta condición y muestra “SIN OBJETO”. Además, se limita el rango de trabajo práctico a 100 cm para evitar presentar como medición válida valores alejados de la zona experimental definida para la práctica.

8. Resultados obtenidos

Durante las pruebas se observó que la lectura cambia cuando se modifica la posición del objeto frente al sensor. El sistema entrega los valores en centímetros y permite comprobar

el comportamiento de manera inmediata mediante el Monitor Serial.

Cuadro 3: Ejemplos de resultados observados durante el proceso.

Condición	Salida esperada	Interpretación
Objeto muy cercano	0.00 cm	Zona de tolerancia cercana
Objeto aproximadamente a 8–12 cm	Valor en cm	Medición normal
Objeto a mayor distancia dentro del rango	Valor en cm	Medición normal
Sin eco válido	SIN OBJETO	No hay lectura válida

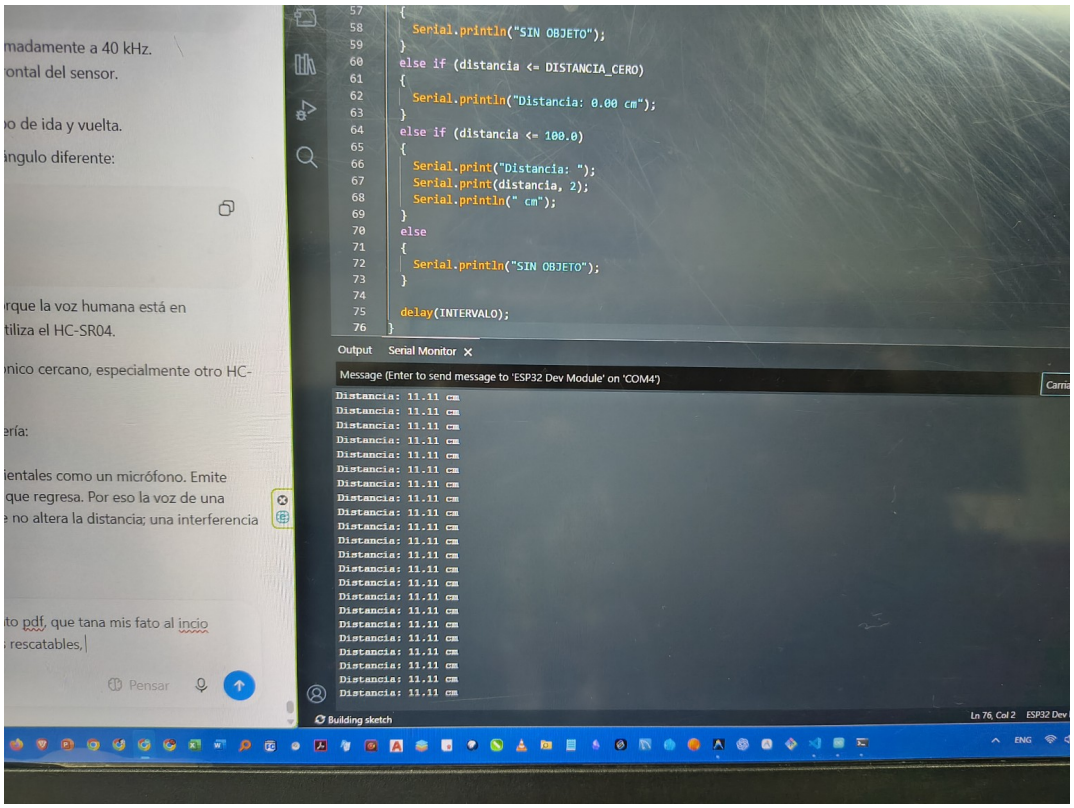


Figura 10: Resultado final observado en el Monitor Serial: lecturas repetidas de 11.11 cm durante la prueba.

9. Problemas encontrados y correcciones

Durante el desarrollo se presentaron dificultades que fueron corregidas progresivamente:

1. **Errores al compilar:** se produjeron errores por copiar texto de explicación y delimitadores de código dentro del Arduino IDE. Se corrigió dejando únicamente el código C++ dentro del sketch.
2. **Problemas de carga al ESP32:** cuando el puerto no entraba correctamente en modo de programación, se utilizó el botón BOOT durante el proceso de carga.
3. **Lecturas inesperadamente grandes o inválidas:** se revisaron las conexiones de TRIG y ECHO y se estableció un tiempo máximo de espera para evitar bloqueos.

4. **Lectura cuando el objeto estaba demasiado cerca:** se añadió la condición `DISTANCIA_CERO` para interpretar una distancia menor o igual a 3 cm como 0.00 cm.
5. **Lecturas sin objeto:** se incorporó la condición de eco inválido y un límite práctico de 100 cm, mostrando "SIN OBJETO" cuando la lectura no es utilizable.

10. Conclusiones

1. Se logró construir y poner en funcionamiento un sistema de medición de distancia usando un ESP32 y un HC-SR04.
2. La conexión principal utilizada fue TRIG en GPIO 5 y ECHO en GPIO 18, además de VCC y GND.
3. El programa calcula la distancia a partir del tiempo de vuelo del pulso ultrasónico y la presenta en centímetros.
4. La comparación con una regla permitió verificar físicamente el orden de magnitud de las mediciones obtenidas.
5. La incorporación de condiciones para distancia mínima, ausencia de eco y rango máximo hizo que la salida del sistema fuera más estable para la práctica.

11. Anexo: referencia visual del montaje

La siguiente referencia permite relacionar las conexiones utilizadas en el programa con los pines físicos del ESP32. Se conserva como material de apoyo para repetir el montaje.

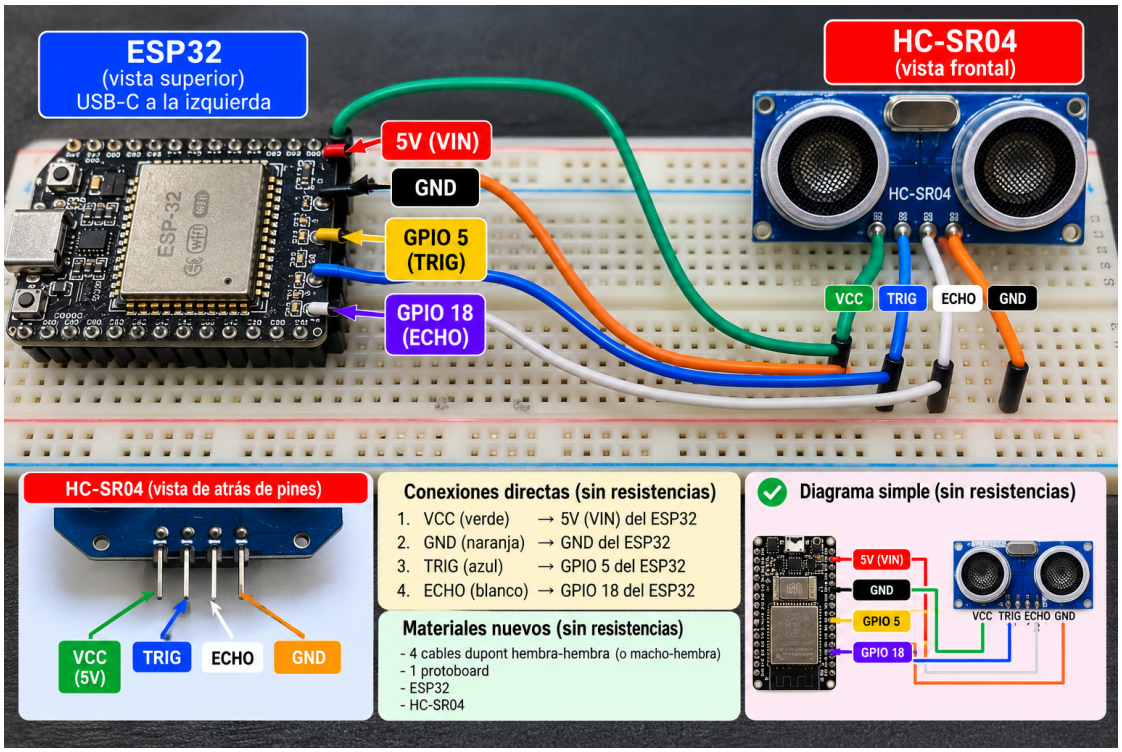


Figura 11: Referencia visual de las conexiones ESP32–HC-SR04.